

Serverless Architectures TASK – 4

1.Create an AWS Lambda function in Python that adds a new item to a DynamoDB table called 'MusicPlaylists' every time it receives a song name and artist via a test event.

Ans :

```
import boto3
```

```
import uuid
```

```
dynamodb = boto3.resource("dynamodb")
```

```
table = dynamodb.Table("MusicPlaylists")
```

```
def lambda_handler(event, context):
```

```
    song_name = event["songName"]
```

```
    artist = event["artist"]
```

```
    item = {
```

```
        "id": str(uuid.uuid4()),
```

```
        "songName": song_name,
```

```
        "artist": artist
```

```
    }
```

```
    table.put_item(Item=item)
```

```
    return {
```

```
"statusCode": 200,  
"body": {  
    "message": "Song added successfully",  
    "item": item  
}  
}
```

2.Modify your Lambda function to read all songs from the 'MusicPlaylists' DynamoDB table and return them as a JSON array.

Ans :

```
import boto3
```

```
import json
```

```
dynamodb = boto3.resource("dynamodb")
```

```
table = dynamodb.Table("MusicPlaylists")
```

```
def lambda_handler(event, context):
```

```
    response = table.scan()
```

```
    songs = response.get("Items", [])
```

```
    return {
```

```
        "statusCode": 200,
```

```
        "headers": {
```

```
            "Content-Type": "application/json"
```

```
},  
    "body": json.dumps(songs)  
}
```

3. Set up Amazon CloudWatch Logs for your Lambda function and trigger it with a test event; then, view the logs to confirm that the song name and artist are correctly logged each time the function runs.
Hint: Use the `print()` statement in Python to log details, and check the CloudWatch Logs console.

Ans :

1. Modify the Lambda function

```
import boto3
```

```
import json
```

```
import uuid
```

```
dynamodb = boto3.resource("dynamodb")
```

```
table = dynamodb.Table("MusicPlaylists")
```

```
def lambda_handler(event, context):
```

```
    song_name = event["songName"]
```

```
    artist = event["artist"]
```

```
    # Log song details to CloudWatch
```

```
    print(f"Song name: {song_name}")
```

```
    print(f"Artist: {artist}")
```

```
item = {
    "id": str(uuid.uuid4()),
    "songName": song_name,
    "artist": artist
}

table.put_item(Item=item)

print(f"Song added successfully: {song_name} by {artist}")

return {
    "statusCode": 200,
    "body": json.dumps({
        "message": "Song added successfully",
        "songName": song_name,
        "artist": artist
    })
}
```

2. Create a test event

In the **AWS Lambda Console**:

1. Open your Lambda function.
2. Select **Test** → **Create new event**.
3. Give it a name such as AddSongTest.

4. Use:

```
{  
  "songName": "Blinding Lights",  
  "artist": "The Weeknd"  
}
```

5. Click **Save** and then **Test**.

You should get a successful Lambda response.

3. View the CloudWatch logs

Go to:

AWS Console → Lambda → Your Function → Monitor → View CloudWatch logs

Open the latest **log stream**. You should see entries similar to:

Song name: Blinding Lights

Artist: The Weeknd

Song added successfully: Blinding Lights by The Weeknd

Every time you run the Lambda with a different test event, the corresponding song name and artist will be written to CloudWatch Logs.

For example:

```
{  
  "songName": "Shape of You",  
  "artist": "Ed Sheeran"  
}
```

will produce:

Song name: Shape of You

Artist: Ed Sheeran

Song added successfully: Shape of You by Ed Sheeran

4. Simulate a bug by intentionally passing an event without the 'artist' field to your Lambda function, then use CloudWatch Logs to debug and fix the error so that the function handles missing fields gracefully.

Ans :

1. Simulate the bug

With the current Lambda code, use this test event:

```
{  
  "songName": "Blinding Lights"  
}
```

Because artist is missing, this line:

```
artist = event["artist"]
```

causes a `KeyError: 'artist'`.

In **CloudWatch Logs**, you should see an error similar to:

```
KeyError: 'artist'
```

2. Fix the Lambda function

Use `.get()` and validate both fields before accessing them:

```
import boto3
```

```
import json
```

```
import uuid
```

```
dynamodb = boto3.resource("dynamodb")
```

```
table = dynamodb.Table("MusicPlaylists")
```

```
def lambda_handler(event, context):

    song_name = event.get("songName")
    artist = event.get("artist")

    # Log received values
    print(f"Song name: {song_name}")
    print(f"Artist: {artist}")

    # Handle missing fields gracefully
    if not song_name or not artist:
        print("ERROR: songName and artist are required")

    return {
        "statusCode": 400,
        "body": json.dumps({
            "error": "songName and artist are required"
        })
    }

item = {
    "id": str(uuid.uuid4()),
    "songName": song_name,
    "artist": artist
```

```
}
```

```
table.put_item(Item=item)
```

```
print(f"Song added successfully: {song_name} by {artist}")
```

```
return {  
    "statusCode": 200,  
    "body": json.dumps({  
        "message": "Song added successfully",  
        "songName": song_name,  
        "artist": artist  
    })  
}
```

3. Test the fix

Run the same invalid event:

```
{  
    "songName": "Blinding Lights"  
}
```

Instead of a Lambda exception, you should now get:

```
{  
    "statusCode": 400,  
    "body": "{\"error\": \"songName and artist are required\"}"  
}
```

CloudWatch should contain:

Song name: Blinding Lights

Artist: None

ERROR: songName and artist are required

Then test with a valid event:

```
{  
  "songName": "Blinding Lights",  
  "artist": "The Weeknd"  
}
```

5. Use ChatGPT to generate a step-by-step checklist for monitoring and debugging a serverless workflow that integrates Lambda with DynamoDB, then apply one suggestion from the checklist to improve your Lambda function's reliability.

Ans :

- 1. Check CloudWatch Logs** — use `print()` or Python's logging module to record important events, inputs, and errors.
- 2. Validate input** — check that required fields such as `songName` and `artist` exist before processing the event.
- 3. Handle exceptions** — use `try/except` around DynamoDB operations so failures are logged and handled cleanly.
- 4. Monitor Lambda metrics** — check invocations, errors, duration, and throttles in CloudWatch.
- 5. Monitor DynamoDB metrics** — watch for throttling, consumed capacity, and request failures.

6. **Use CloudWatch alarms** — create alarms for unusually high Lambda errors or DynamoDB throttling.
7. **Test failure cases** — deliberately test missing fields, invalid data, and DynamoDB failures.
8. **Check IAM permissions** — verify that the Lambda execution role has the required DynamoDB and CloudWatch permissions.
9. **Use structured logging** — include useful context such as the operation, song name, and error message.
10. **Review logs after deployment** — confirm both successful requests and failures appear as expected.

```
import boto3
import json
import uuid
from botocore.exceptions import ClientError

dynamodb = boto3.resource("dynamodb")
table = dynamodb.Table("MusicPlaylists")

def lambda_handler(event, context):
    song_name = event.get("songName")
    artist = event.get("artist")

    print(f'Received song: {song_name}, artist: {artist}')

    # Validate input
```

```
if not song_name or not artist:
```

```
    print("ERROR: songName and artist are required")
```

```
    return {
```

```
        "statusCode": 400,
```

```
        "body": json.dumps({
```

```
            "error": "songName and artist are required"
```

```
        })
```

```
    }
```

```
item = {
```

```
    "id": str(uuid.uuid4()),
```

```
    "songName": song_name,
```

```
    "artist": artist
```

```
}
```

```
try:
```

```
    table.put_item(Item=item)
```

```
    print(f"Song added successfully: {song_name} by {artist}")
```

```
    return {
```

```
        "statusCode": 200,
```

```
        "body": json.dumps({
```

```
            "message": "Song added successfully",
```

```
        "item": item
    })
}
```

```
except ClientError as error:
```

```
    print(f'DynamoDB error: {error}')
```

```
    return {
        "statusCode": 500,
        "body": json.dumps({
            "error": "Unable to save song"
        })
    }
```